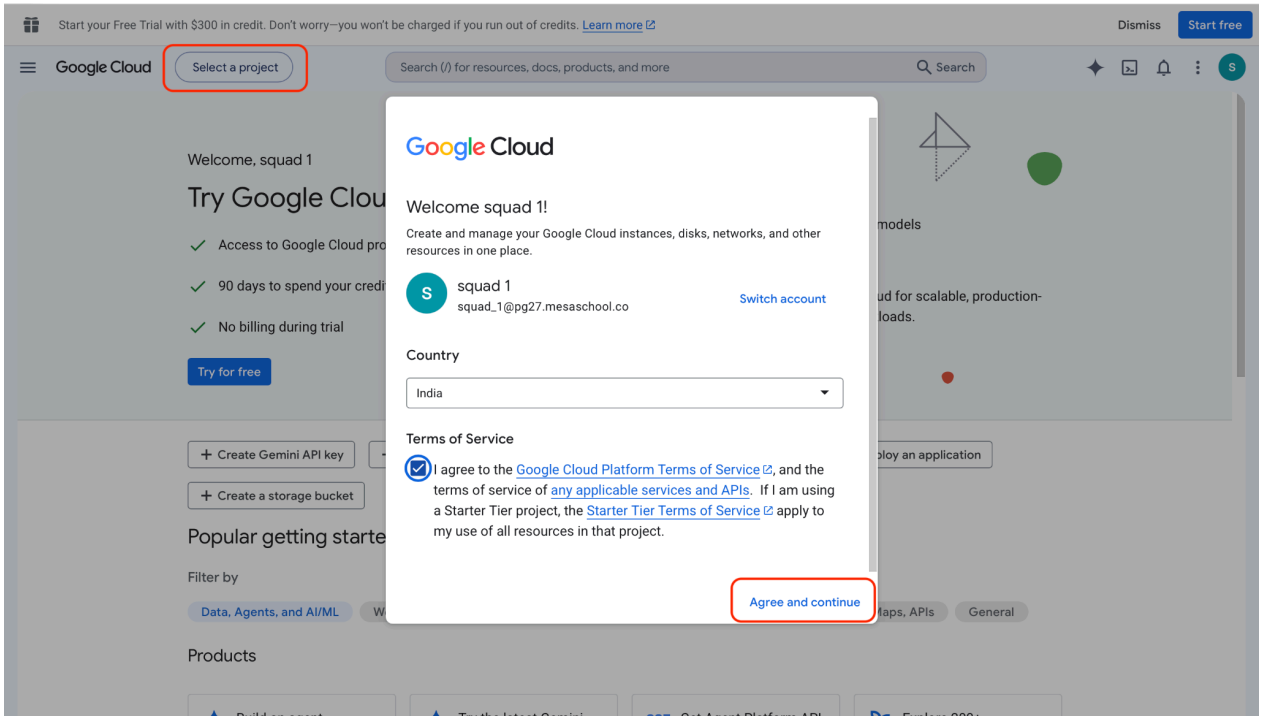
Setting Up Google OAuth

Your Telegram bot will need to read your Google Drive files, fetch your Gmail threads, check your Calendar, and write to Google Sheets. To do this, you must grant it access using Google OAuth 2.0. Complete the steps below before the session begins - it takes about 15–20 minutes.

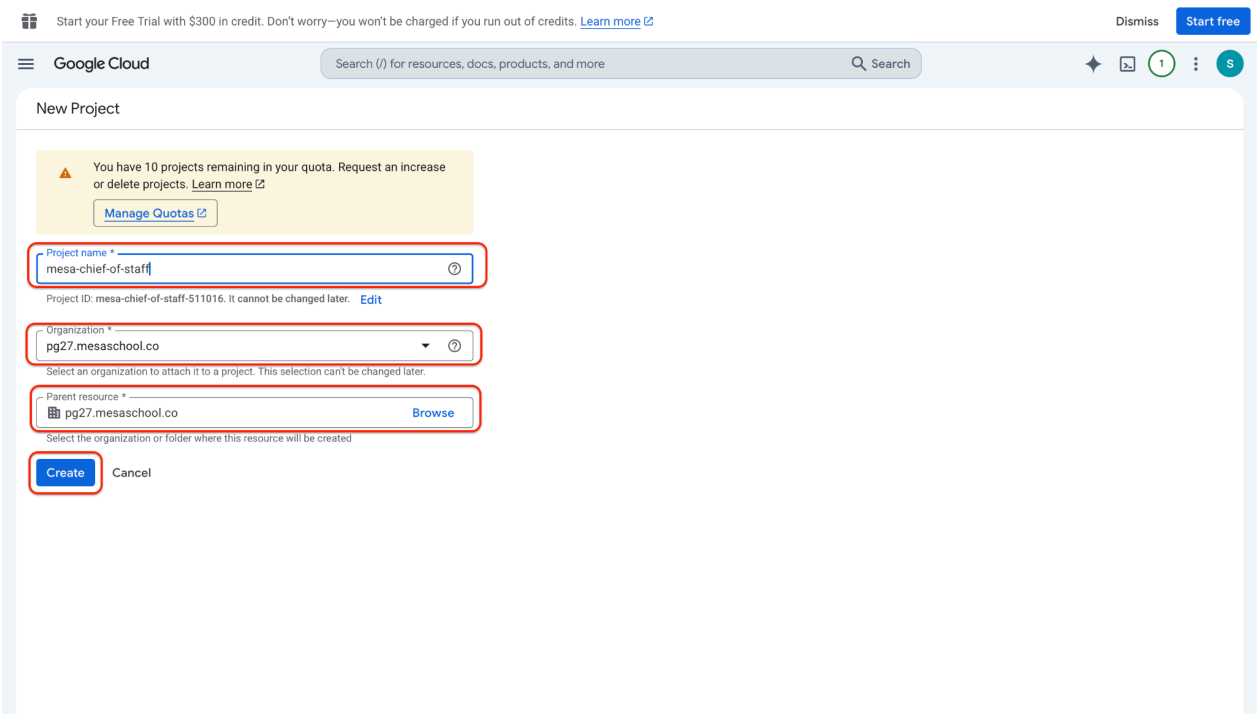
Part 1: Create a GCP Project

1. Go to console.cloud.google.com and sign in with your @pg27.mesaschool.co account.



2. Click the project selector at the top of the page, then select New Project.
   If this is your first time, the selector will say "Select a project".

3. Give your project a name - for example, mesa-chief-of-staff - and click Create.

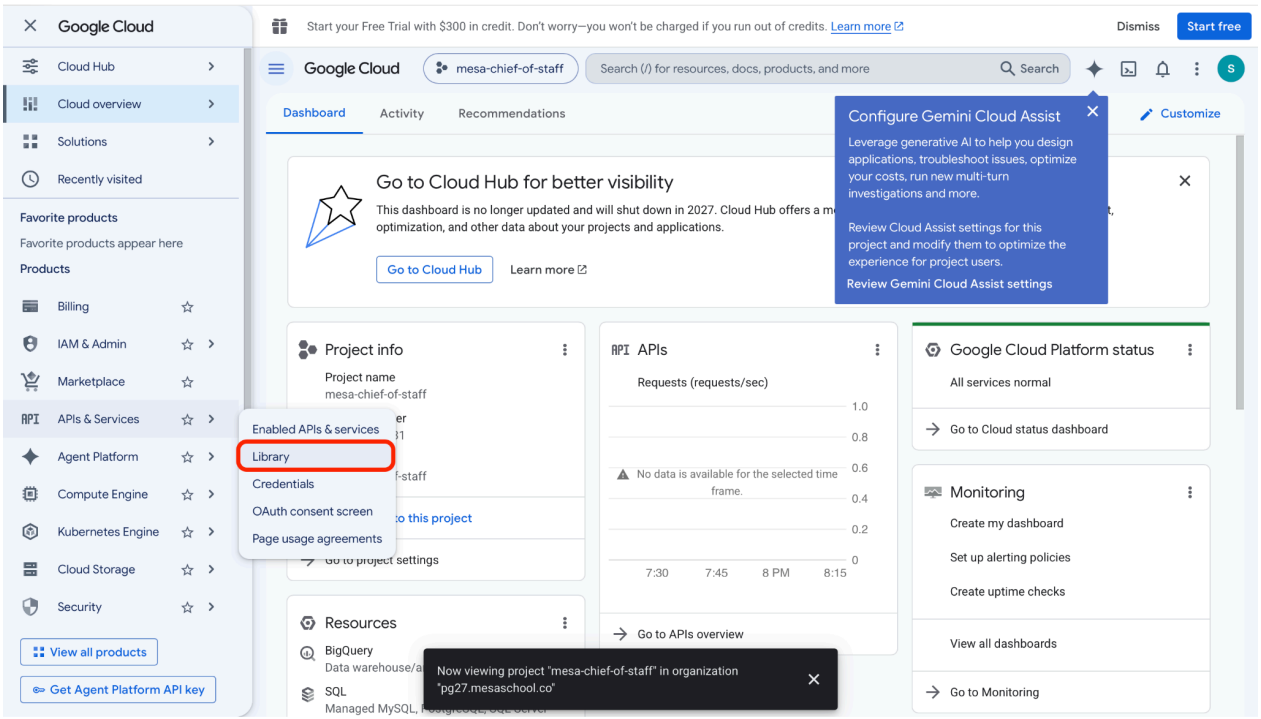


4. Wait for the project to be created, then make sure it is selected in the project selector before proceeding.

Part 2: Enable the Required APIs

---

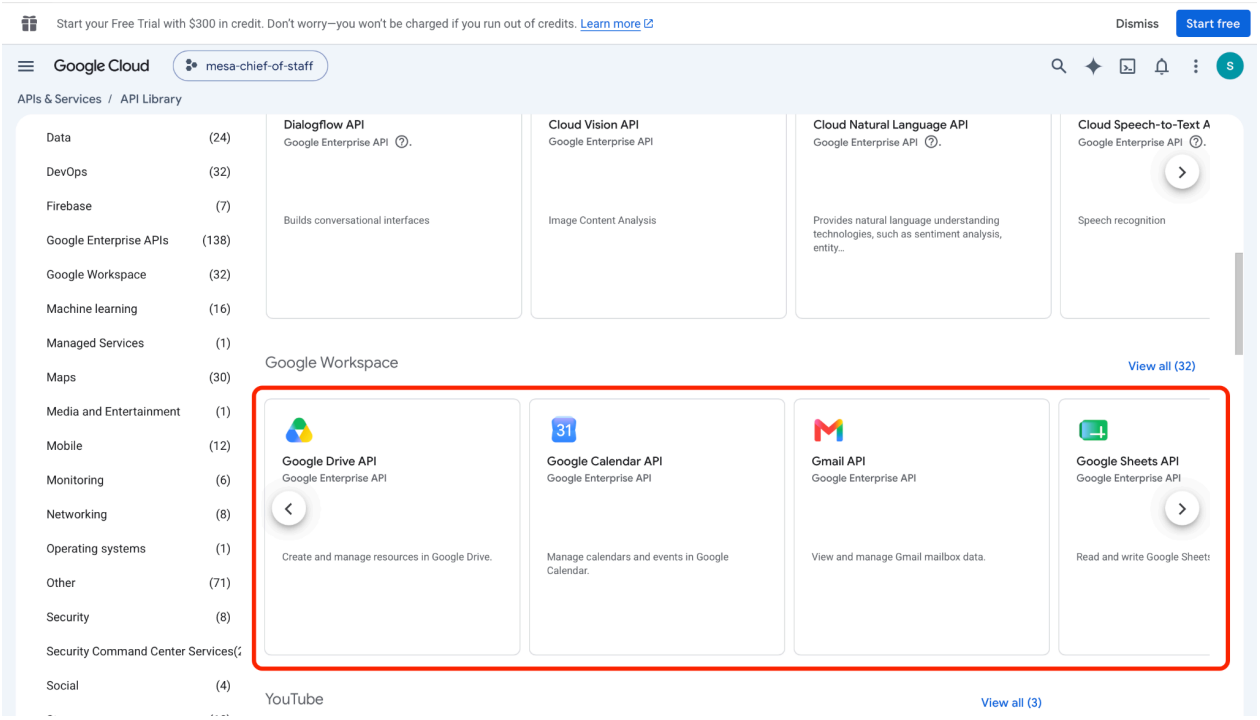
1. In the left sidebar, navigate to APIs & Services → Library.

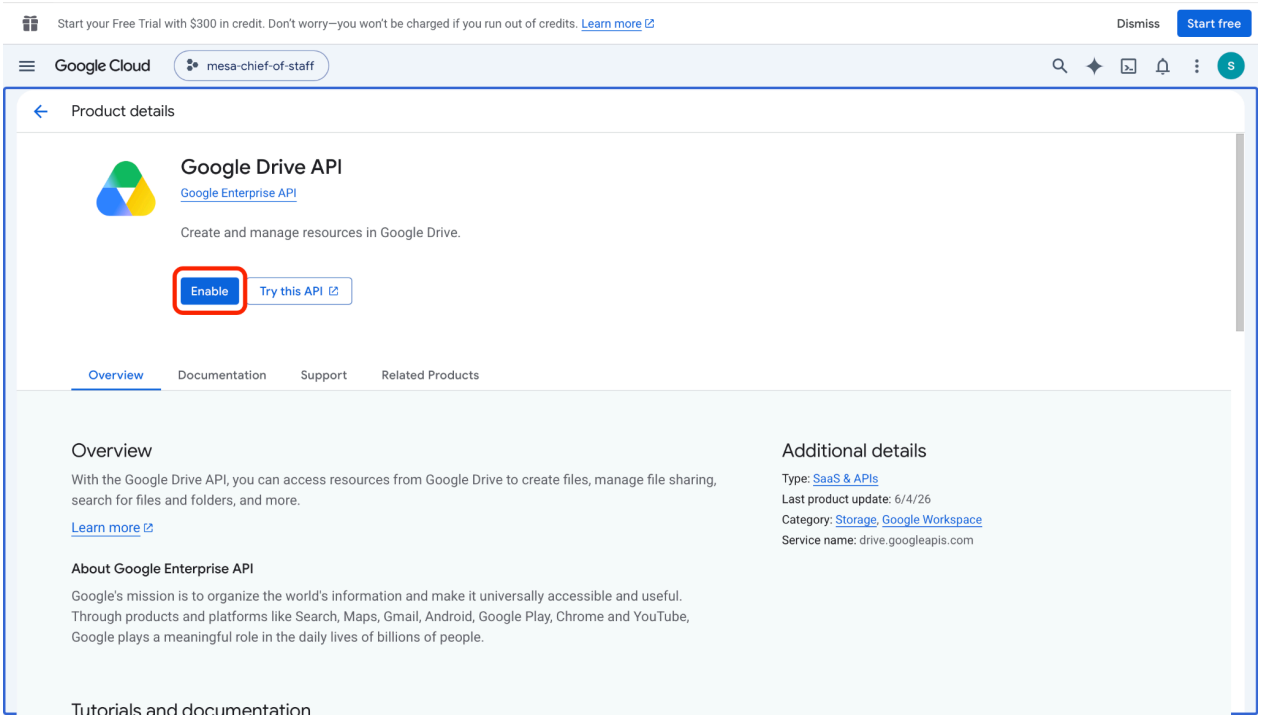


2. Search for and enable each of the following APIs one at a time:

- Google Drive API

- Gmail API
- Google Calendar API
- Google Sheets API

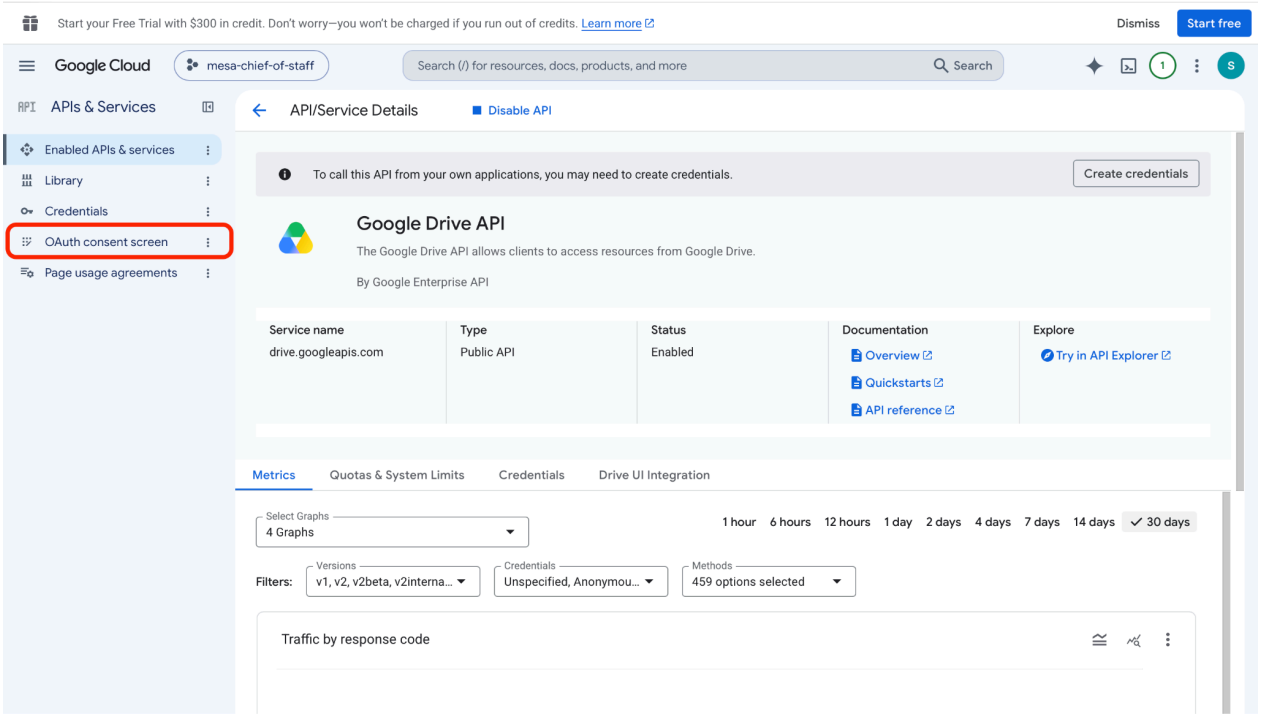


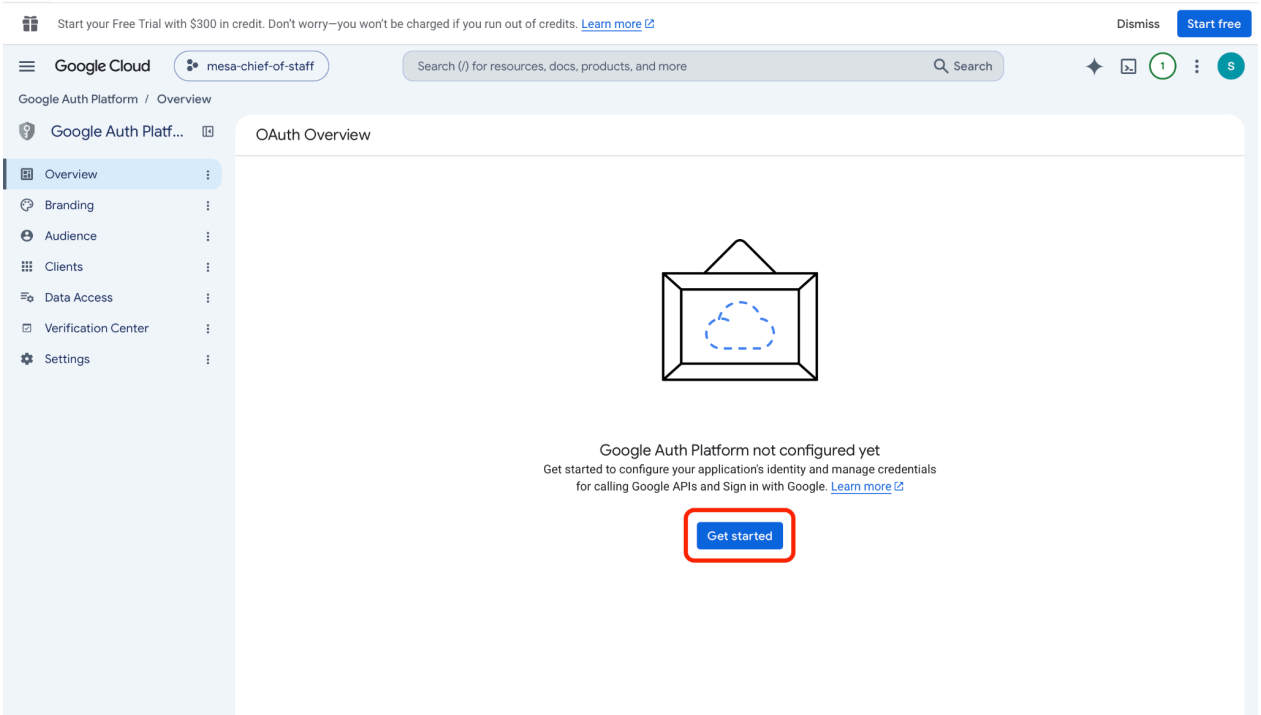


Part 3 — Configure the OAuth Consent Screen

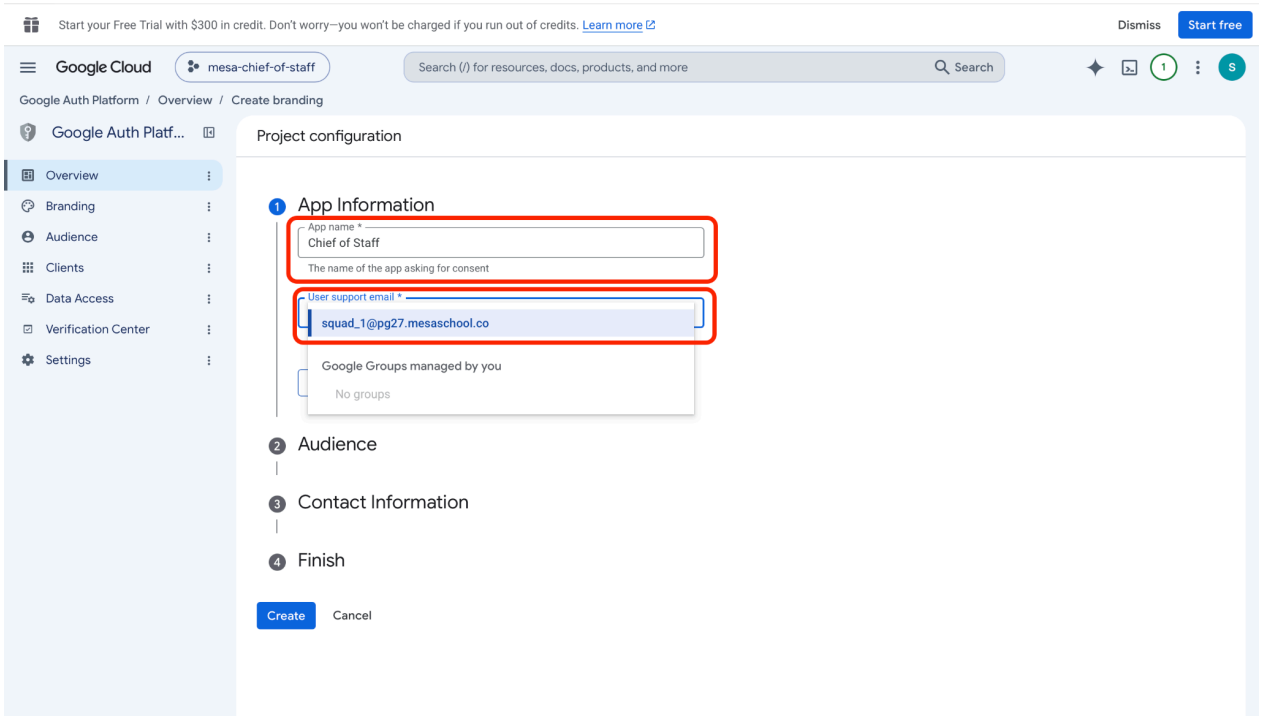
1. In the sidebar, go to APIs & Services → OAuth consent screen.

> **IMPORTANT:** When asked to choose a User Type, select Internal — not External. This restricts access to @pg27.mesaschool.co accounts only and keeps your data private. If you select External by mistake, anyone with a Google account could request access.

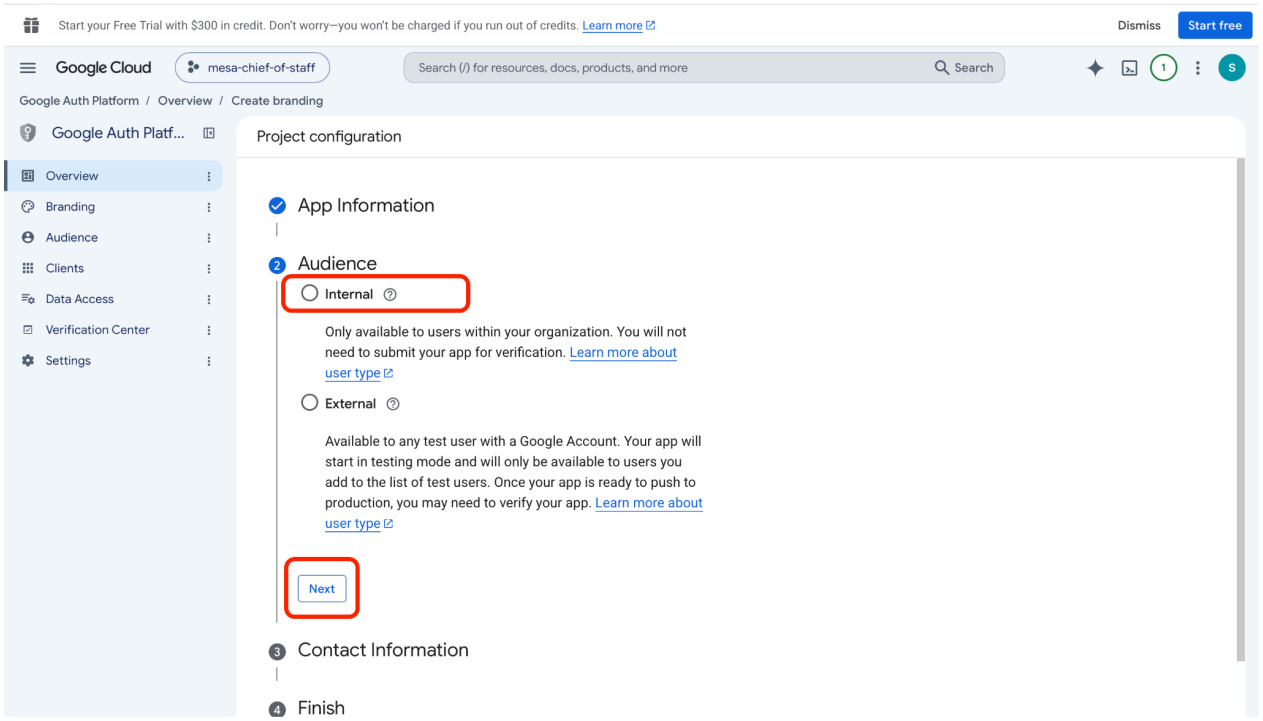


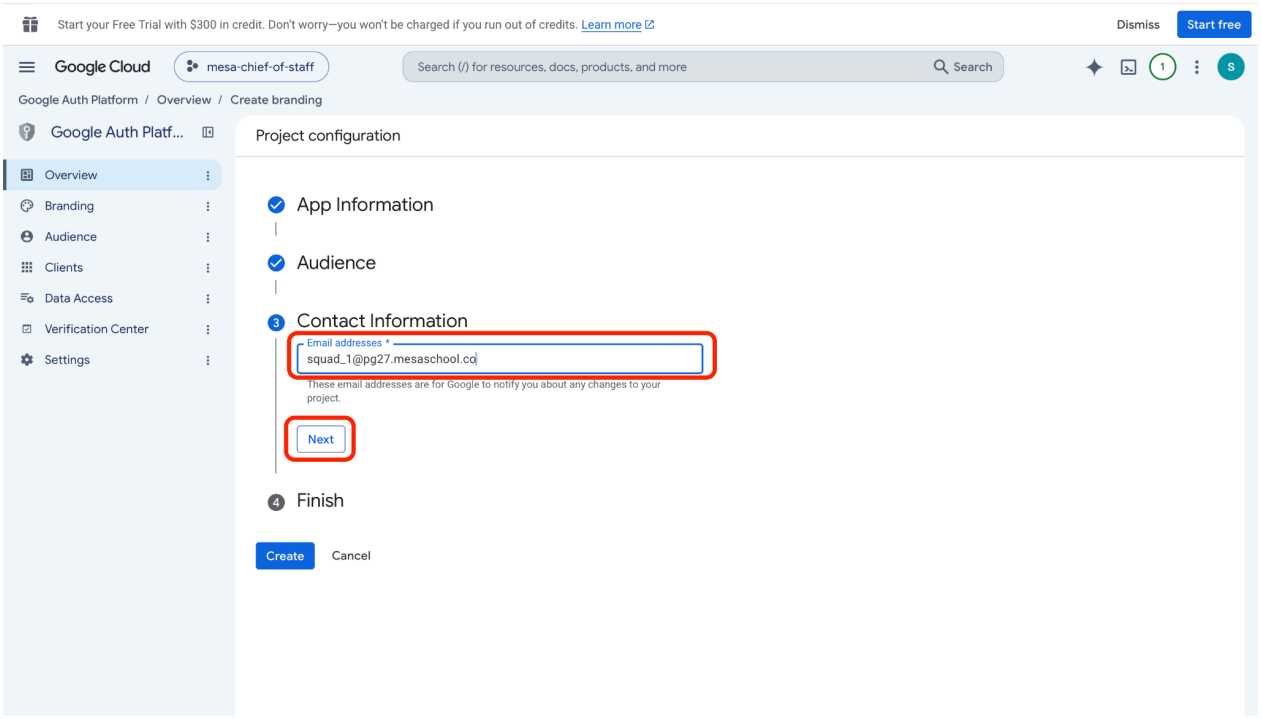


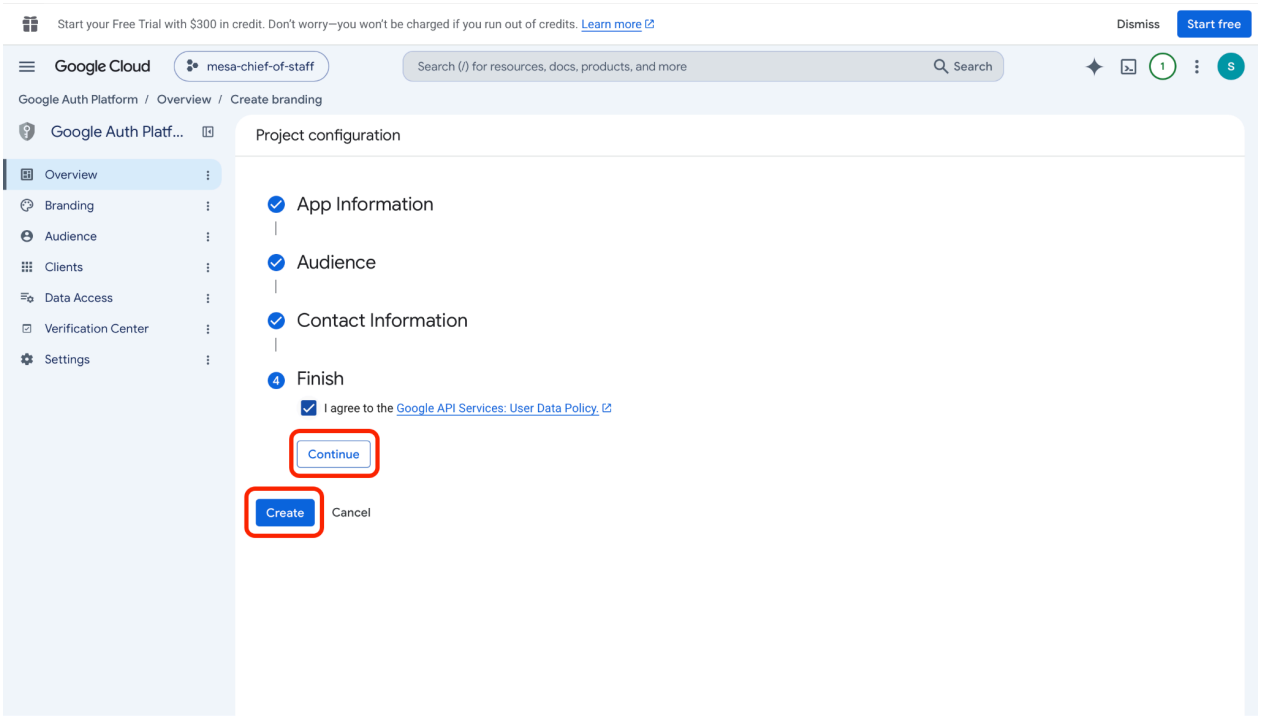
2. Click Create. Fill in the App name (e.g. Chief of Staff), and set the User support email and Developer contact email to your @pg27.mesaschool.co address.



3. IMPORTANT: When asked to choose a User Type, select Internal — not External. This restricts access to @pg27.mesaschool.co accounts only and keeps your data private. If you select External by mistake, anyone with a Google account could request access.





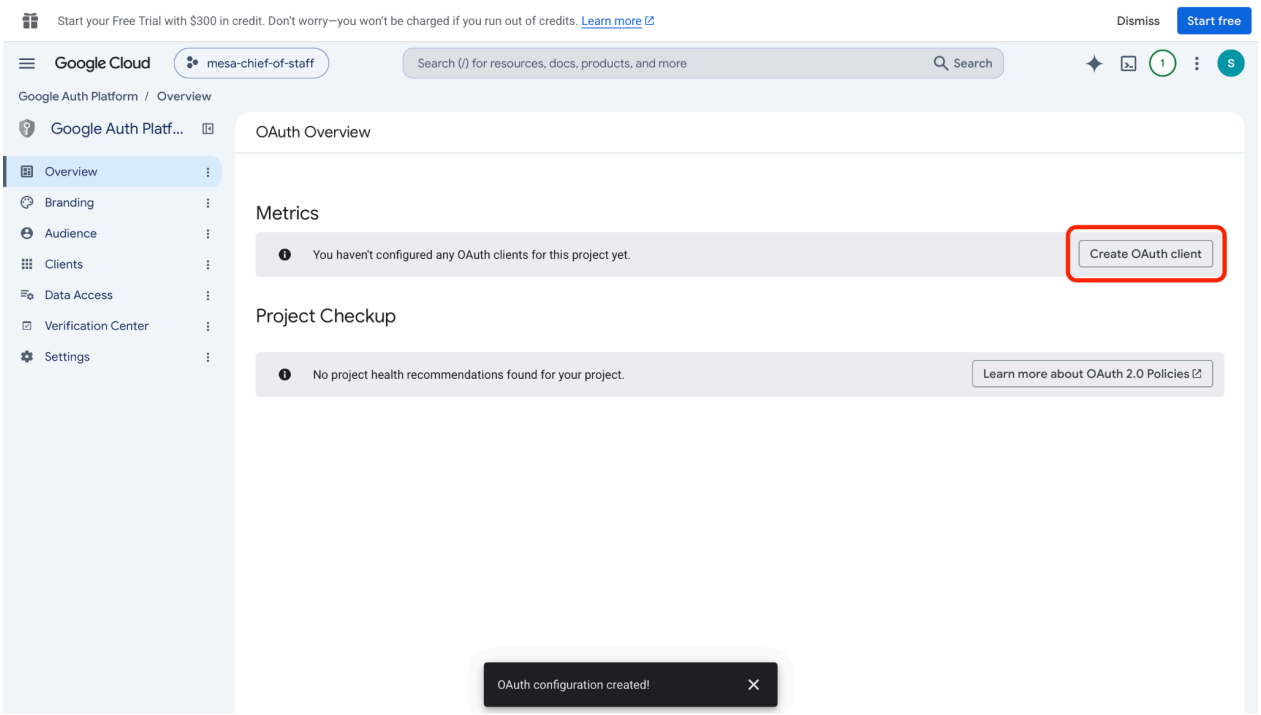


4. Click Save and Continue to move to the next step.

Part 4: Create OAuth Credentials

---

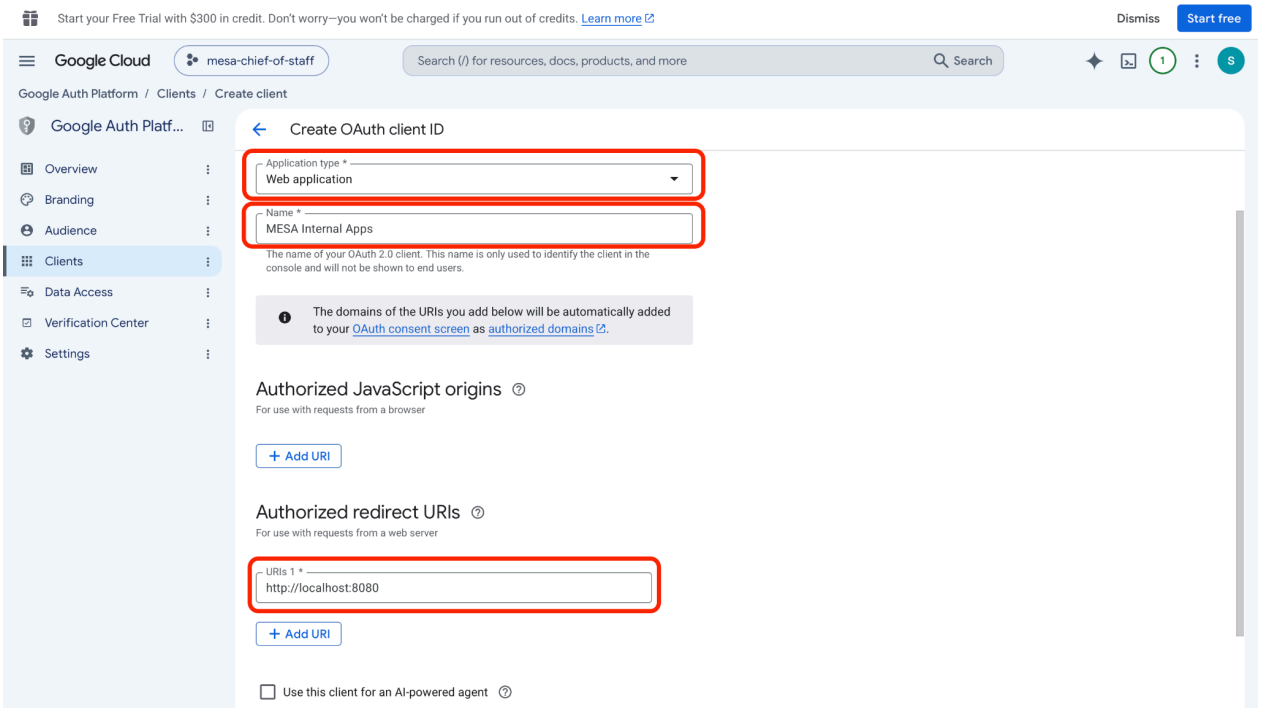
1. If you are not automatically redirected to the page below, go to 'APIs & Services → Credentials' and click 'Create Credentials → OAuth client ID'.



2. Set Application type to Web application. Give it any name.
3. Under Authorised redirect URIs, add:

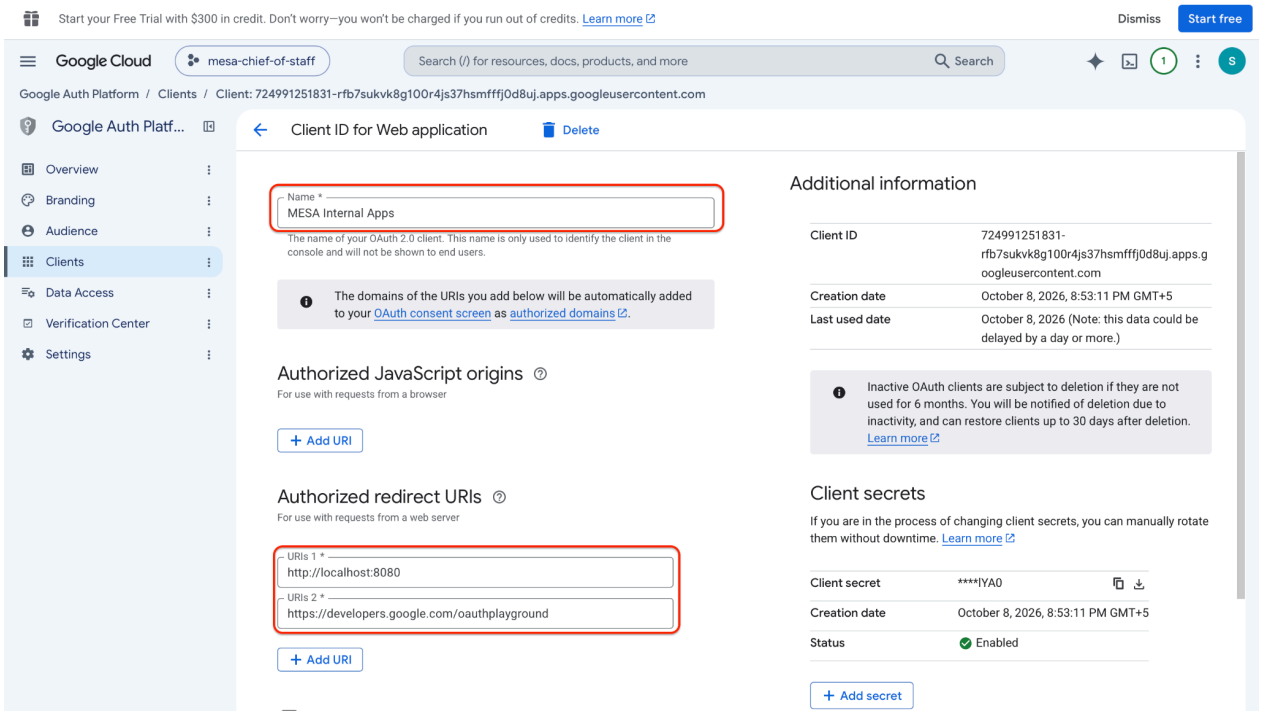
```
http://localhost:8080
```

This is only used once to complete the authorisation flow.



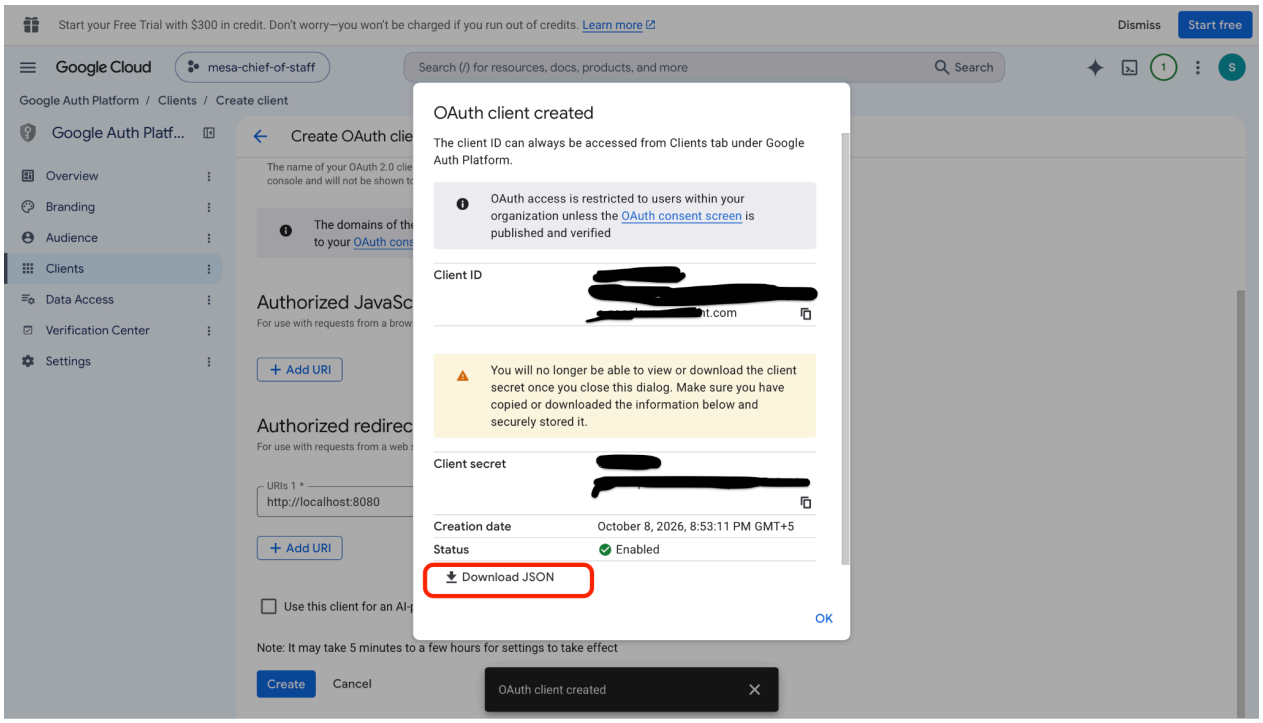
4. Under Authorised redirect URIs, add another uri:

```
https://developers.google.com/oauthplayground
```



4. Click Create. A dialog will show your Client ID and Client Secret. Click Download JSON to save the credentials file.

Keep this file safe — do not commit it to GitHub.



Part 5: Generate a Refresh Token

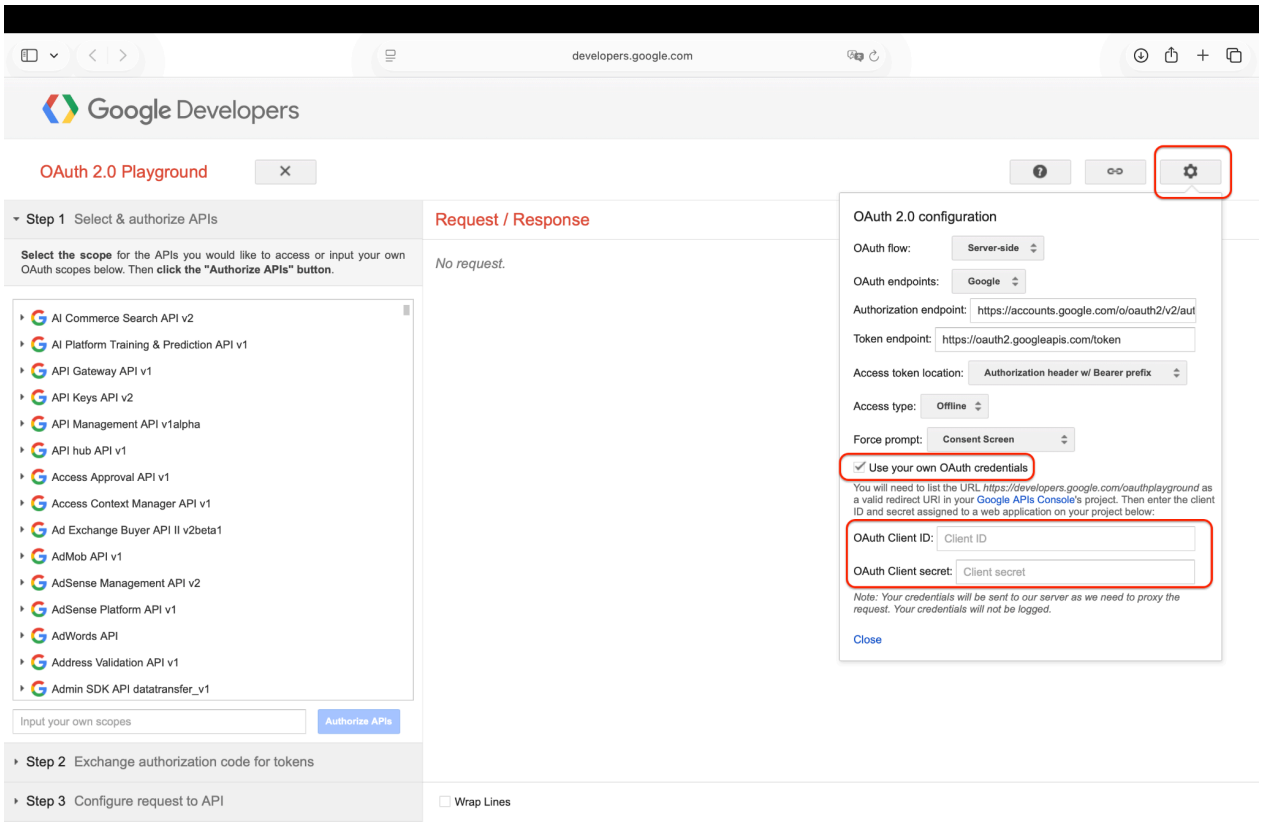
You need to run the OAuth flow once to get a refresh token. Your Vercel app will use this token to obtain fresh access tokens without asking you to log in again.

1. Go to the OAuth Playground

Open: https://developers.google.com/oauthplayground

2. Use your own credentials

Click the ⚙️ gear icon (top right of the page). A panel opens. Check **"Use your own OAuth credentials"** and paste in your **Client ID** and **Client Secret** from GCP. Close the panel.



3. Select your scopes

In the left panel of the above image, scroll to find and tick these five (or paste them into the input box at the bottom of the list):

```
None
https://www.googleapis.com/auth/drive.readonly
https://www.googleapis.com/auth/gmail.readonly
https://www.googleapis.com/auth/gmail.send
https://www.googleapis.com/auth/calendar
https://www.googleapis.com/auth/spreadsheets
```

Click **Authorise APIs**.

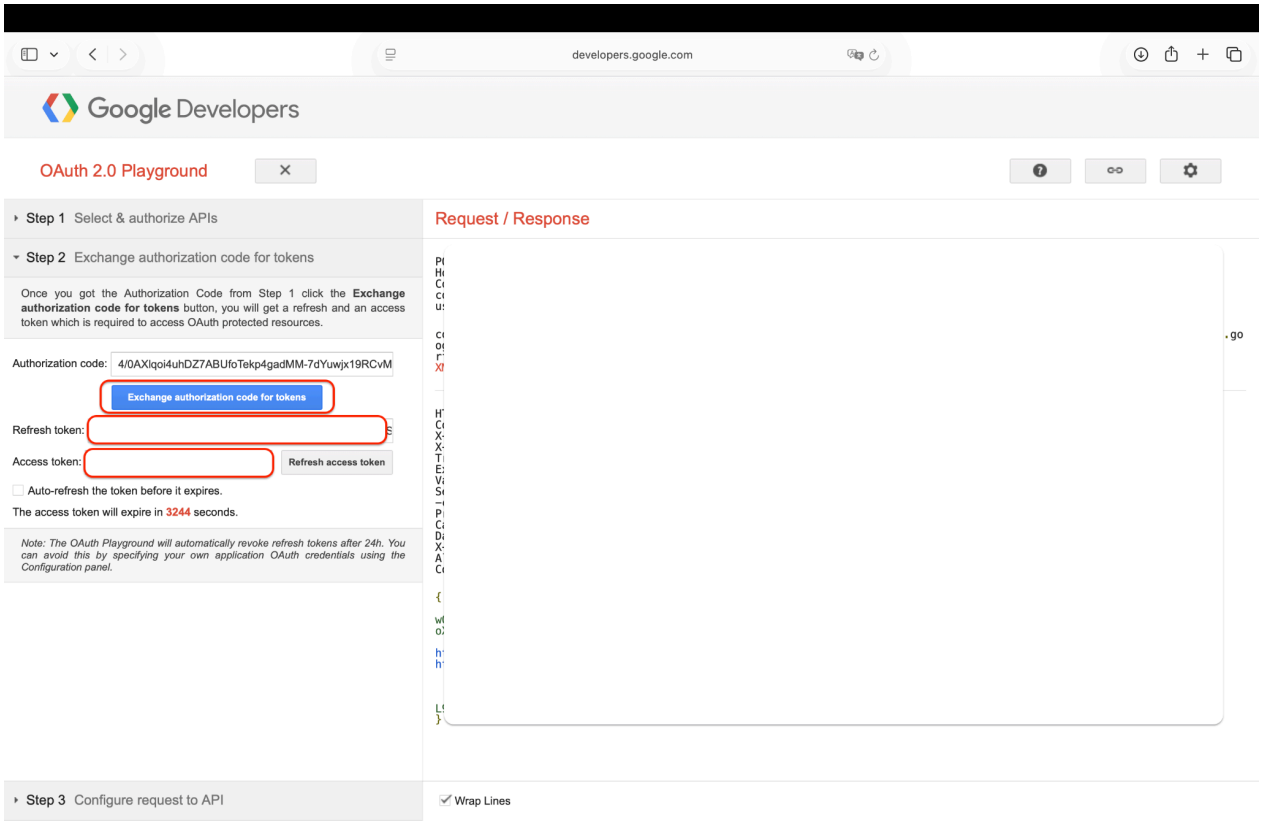
4. Sign in and approve

A Google login screen appears. Sign in with your `@pg27.mesaschool.co` account. Click **Allow** on the permissions screen.

5. Get the Refresh Token

Click **Exchange authorisation code for tokens**.

Your **Refresh Token** appears in the response box on the right. Copy it. Combined with your Client ID and Client Secret from GCP, you now have all three values the bot needs.



4. Your app will require 3 values - save them securely:

```
CLIENT_ID
CLIENT_SECRET
REFRESH_TOKEN
```

> **IMPORTANT:** Do not share these values or paste them into any chat, doc, or screenshot. Treat them like passwords - they grant full access to your Drive, Gmail, Calendar, and Sheets.

Part 6: Add to Vercel Environment Variables

---

1. Open your Vercel project dashboard and go to Settings → Environment Variables.
2. Add each of the following as a separate environment variable:

```
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
GOOGLE_REFRESH_TOKEN
```

3. Click Save. Then go to your Vercel project's Deployments tab and redeploy to pick up the new variables.

After redeployment, your app can authenticate to Google without any further action from you.

Quick Check

Before the session begins, confirm each of the following:

✓	What to verify
☐	GCP project created and selected in the console
☐	All four APIs enabled (Drive, Gmail, Calendar, Sheets)
☐	OAuth consent screen set to Internal — not External
☐	All five scopes added (drive.readonly, gmail.readonly, gmail.send, calendar, spreadsheets)
☐	OAuth client ID created and credentials JSON downloaded
☐	Refresh token generated successfully using the helper script
☐	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, and GOOGLE_REFRESH_TOKEN added to Vercel
☐	Vercel redeployed after adding the environment variables